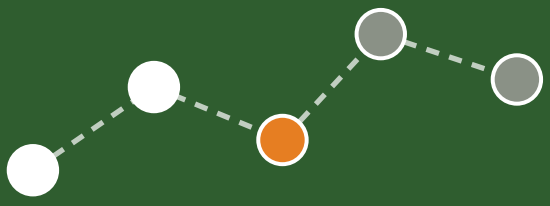
Birdify

From inspiration to reality: designing, building and testing a cross-platform bird-discovery game



Technical Report

Project	Interaction Programming — VT26, KTH Royal Institute of Technology
Platform	React Native + Expo (Android, iOS, Web)
Stack	MobX · Firebase Auth & Firestore · Google Maps · Wikipedia REST · xeno-canto
Team	Birdify project group

Abstract. **Birdify** is an educational game that turns bird identification into a journey across the world's continents. Players pick a continent, follow a route of waypoints on an interactive map, and identify the bird waiting at each stop from its photograph and its recorded song. Each new species earns a feather, every third discovery earns an egg, and milestones unlock badges. This report traces the project from its first sources of inspiration and hand-drawn sketches, through the architecture and the external APIs it relies on, to the functional and user testing that shaped the final product. Two rounds of user testing (one participant on the early prototype, three on the final build) drove concrete changes, including a badge-based achievements view, one-bird-at-a-time challenges, and clearer navigation. Mean layout-intuitiveness rose from 3.0 to 4.3 out of 5.

Contents

1 Introduction **2**

2 Design Inspiration **2**

2.1 National birds on a map . . . 2

3 Design Ideas **3**

3.1 First sketch: “Pick your bird world” . . . 3

3.2 Design principles . . . 4

3.3 Visual language . . . 4

4 From Prototype to Final Product **5**

4.1 Prototype 1.0 (March 2026) . . . 5

4.2 Final product (May–June 2026) . . . 5

4.3 Achievements and badges . . . 6

5 System Architecture **7**

5.1 Game rules in code . . . 8

6 APIs and Services **8**

6.1 Fetching the clues . . . 9

6.2 Keys and configuration . . . 9

7 Testing **9**

7.1 API feasibility testing . . . 9

7.2 Cross-platform functional testing . . . 10

7.3 State and persistence testing . . . 10

8 User Testing **10**

8.1 Method . . . 10

8.2 Quantitative results . . . 11

8.3 Qualitative findings . . . 11

8.4 From feedback to features . . . 12

9 Conclusion and Future Work **13**

1 Introduction

Most people can recognise a handful of birds at best, and the tools that teach more, such as field guides and identification apps, assume the learner is already motivated. **Birdify** tries to supply that motivation with a game loop: a sense of place (a real map, real countries), a clear goal (the next stop on the route), immediate feedback (right or wrong, with animation), and something to collect.

The application is built once in React Native with Expo and runs on Android, iOS and the web. Table 1 summarises the scope of the final version.

Table 1. **Birdify** at a glance (final build).

Continents	7	Answer options per question	4
Birds per continent	9	Achievement badges	13
Total species	63	Collectible types	3 (feathers, eggs, birds)
Target platforms	3	External data APIs	2 (Wikipedia, xeno-canto)

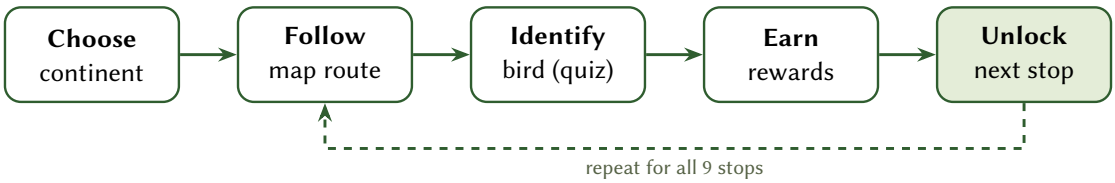


Figure 1. The core game loop of **Birdify**.

2 Design Inspiration

2.1 National birds on a map

The starting point was an infographic of the national birds of European countries (Figure 2). Three things about it stood out to us and carried through into the product:

1. **Birds belong to places.** Every bird is tied to a country and a spot on a map. Knowing *where* a bird lives helps people remember it, so geography became the backbone of the game rather than decoration.
2. **Circular photo tokens.** Round, numbered photographs of birds are easy to scan and feel collectible. They became the discovered-waypoint markers on our map, where a found bird's photo replaces the placeholder pin.
3. **A calm, natural palette.** The lime-green on white of the infographic suggested an outdoor, field-guide mood. We shifted it to a deeper forest green for contrast and accessibility (Section 3.3).



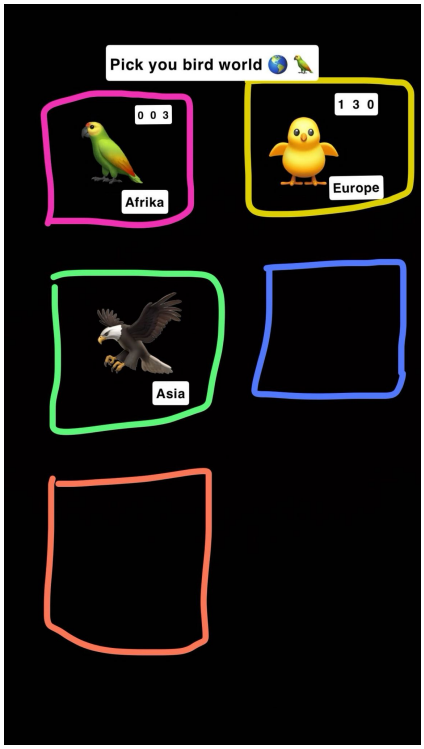
Figure 2. Inspiration: an infographic pairing European countries with their national birds. The number–photo–map pattern shaped **Birdify**'s waypoint markers and continent structure.

> **Design takeaway.** The infographic already has a question-and-answer structure: a numbered photo, and a name you look up in the table. Turning that look-up into a multiple-choice quiz, and the table into a route, gave us the core mechanic.

3 Design Ideas

3.1 First sketch: "Pick your bird world"

The first concept was a quick hand-drawn sketch (Figure 3). The home screen was a grid of colour-coded tiles, one per continent, each with a mascot bird and a three-digit counter showing how many species the player had collected there.



Ideas captured in the sketch

- One tile per continent, each with its own colour and bird mascot.
- A visible per-continent counter (003, 130), which later became *"Birds discovered: n/9"*.
- Empty tiles for continents not yet built. This is where the incremental plan came from (Europe first, then all seven).

What changed on the way to code

- Tiles became full-width cards, which read better on narrow phones and scale to seven continents without cramping.
- Emoji mascots became real bird photographs (Figure 5b).
- The neon-on-black palette was replaced by a light, natural theme.

Figure 3. Early concept sketch of the continent picker.

3.2 Design principles

Place before name
Each question is tied to a country and coordinates, so the player always knows *where* they are before being asked *what* they are looking at.

Two senses, one answer
Every question gives both a photograph (Wikipedia) and a real field recording (xeno-canto). Players who struggle with one clue can rely on the other.

Progress you can see
The route, the lock icons, and the collection counter always show how far the player has come and what comes next.

Gentle rewards
Feathers for every new bird and eggs every third discovery keep rewards frequent early on, while badges give longer-term goals.

3.3 Visual language

The palette is taken directly from the style sheets in `src/views/`. Green signals progress and primary actions, orange marks the current stop, and grey marks locked content.

#2F5D2F #1F2A16 #566246 #D5DFC6 #F5F8EF #E67E22 #8A9186

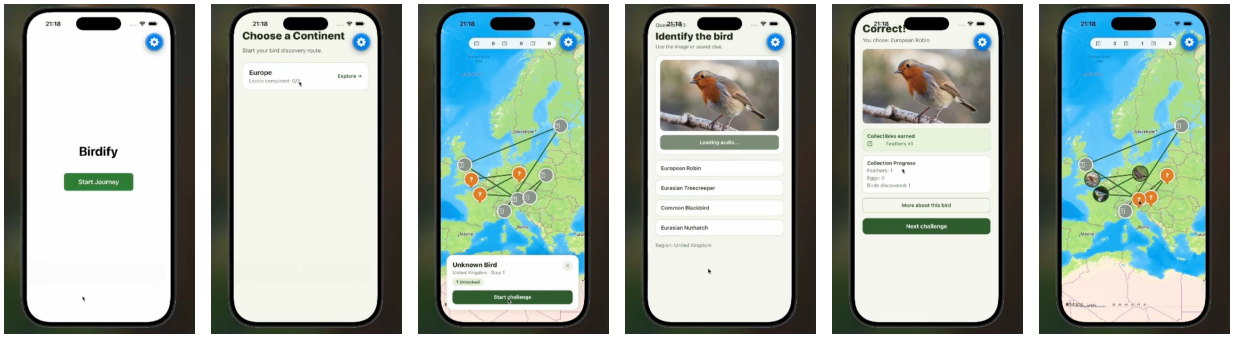
Table 2. Waypoint states on the map and how each is shown.

State	Marker	Meaning and interaction
Locked	grey + padlock	Earlier stops must be completed first; the button reads *"Complete previous stops first"* and is disabled.
Current	orange "?" pin	Unlocked but not yet identified: *"Start challenge"*.
Completed	bird photo	The discovered bird's photo replaces the pin; tapping shows details.

4 From Prototype to Final Product

4.1 Prototype 1.0 (March 2026)

The first working prototype (Figure 4) covered the full loop for a single continent, Europe, on iOS. It already had the map route, the combined image-and-audio quiz, and the reward summary. It was the build used in the first round of user testing.

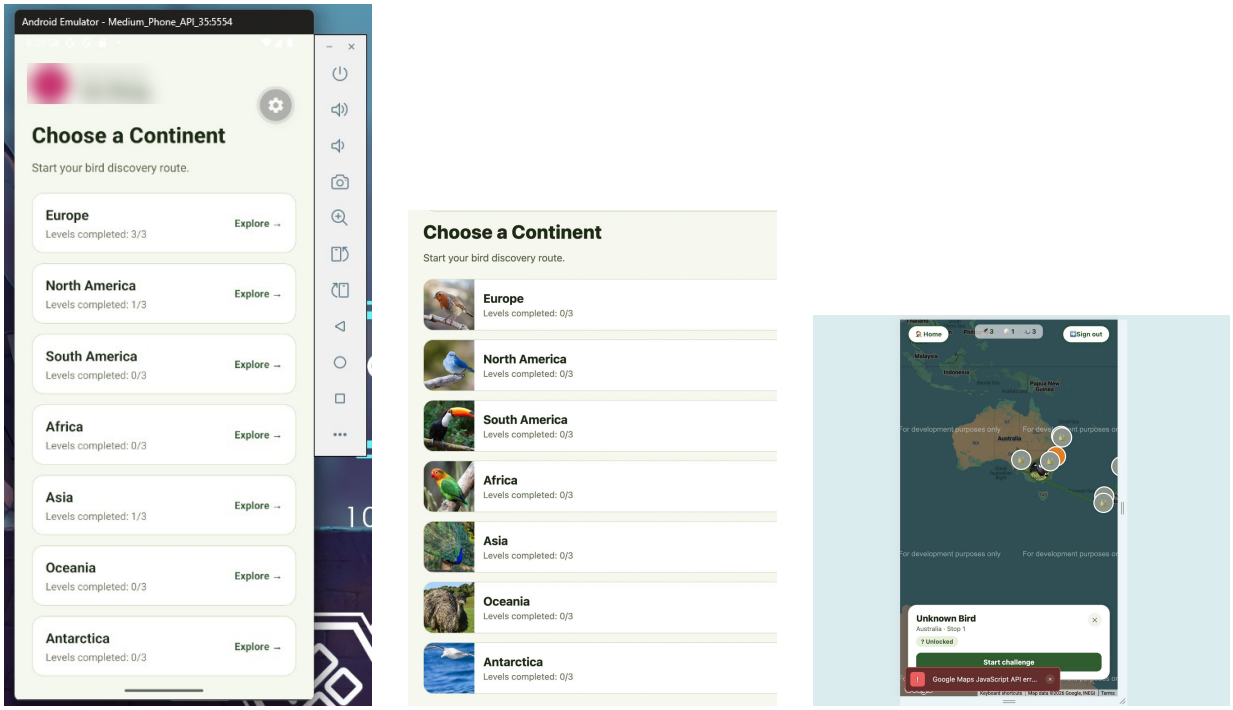


(a) Start **(b)** Continent **(c)** Route **(d)** Quiz **(e)** Result **(f)** Progress

Figure 4. Prototype 1.0 on iOS: (a) splash screen, (b) Europe-only continent list, (c) map with the first stops unlocked, (d) the "Identify the bird" question with photo and audio clue, (e) correct-answer feedback with collectibles earned, (f) the map after three discoveries, where bird photos have replaced the question-mark pins.

4.2 Final product (May–June 2026)

The final version expands to all seven continents, adds authentication and cloud persistence, an onboarding panel, photo-led continent cards, and the achievements system.



(a) Android: all seven continents **(b)** Continent cards with photos **(c)** Web: Oceania route

Figure 5. Final build. (a) An intermediate Android build listing all continents; (b) the final continent cards, each fronted by a representative bird; (c) the web map with padlocked stops, the collectible counter at the top and labelled Home / Sign-out buttons. The user's name is blurred in (a).

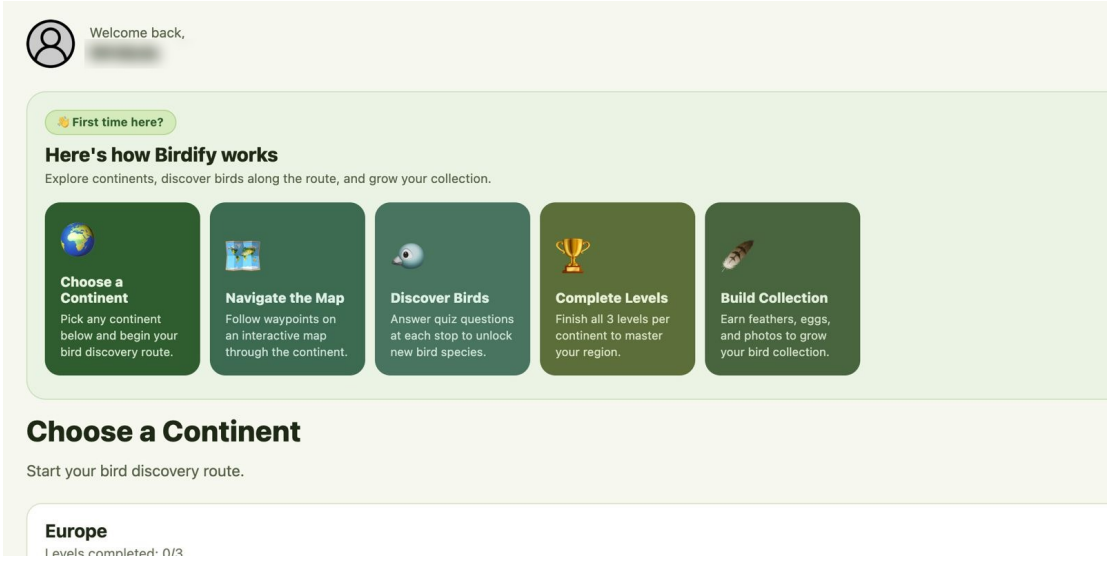


Figure 6. First-run onboarding on the web ("Here's how **Birdify** works"), added in response to first-round feedback that the purpose of the home page was unclear. User name blurred.

4.3 Achievements and badges

Following user feedback (Section 8.4), collectibles gained a dedicated *Achievements* sheet, opened by tapping the collectible counter on the map. It shows the running totals and a grid of 13 badges across three tracks. Unearned badges appear as locked "???" cards so players can see there is more to find.

Table 3. Badge tracks and thresholds (`src/views/achievementsView.jsx`).

Track	Badges (threshold)
Feathers	First Feather (1) · Feather Collector (5) · Feather Fan (10) · Feather Hoarder (25)
Eggs	First Egg (1) · Egg Hunter (5) · Nest Raider (10) · Egg Expert (25)
Birds	First Discovery (1) · Bird Watcher (5) · Ornithologist (10) · Bird Expert (20) · Master Birder (50)



Figure 7. A selection of the badge artwork shown in the Achievements view.

5 System Architecture

Birdify follows a **Model–View–Presenter** structure. A single MobX observable model holds all game state. Presenters (`src/reactjs/`) are MobX *observers* that read the model and pass plain props and callbacks to Views (`src/views/`), which contain no application logic. Expo Router maps files in `src/app/` to screens.

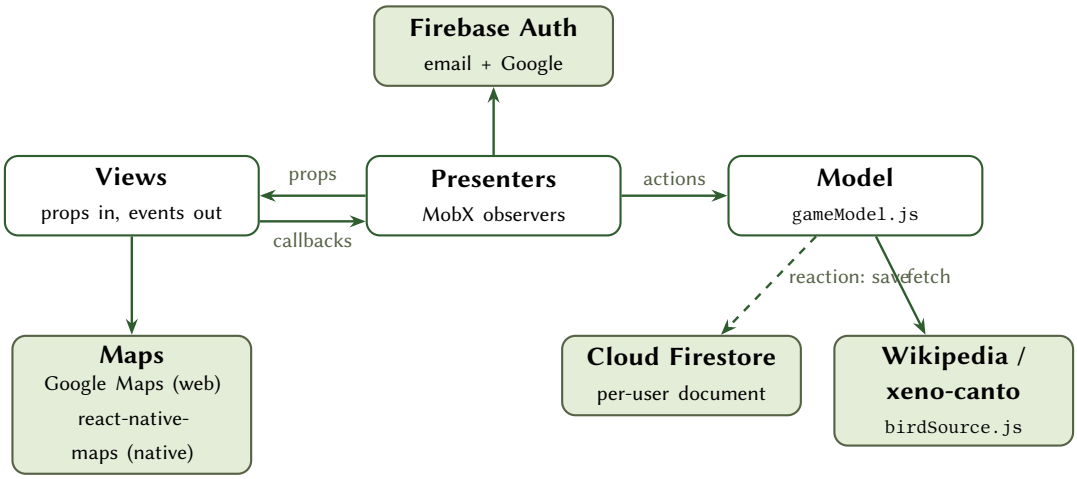


Figure 8. Architecture overview. Solid arrows are calls; the dashed arrow is a MobX reaction that writes to Firestore whenever `progress` or `collection` changes.

Table 4. Key modules.

Module	Responsibility
`gameModel.js`	Observable state and actions: selecting a continent/question, answer checking, reward calculation, unlocking the next stop, building map waypoints.
`birdsLevels.js`	Static content: 63 species with common and scientific names, country, and latitude/longitude; distractor selection.
`birdSource.js`	Wraps the Wikipedia and xeno-canto APIs.
`firestoreModel.js`	Loads the user's document on login and saves changes through a MobX reaction.
`authService.js`	Email sign-in (with verification check), registration and Google Sign-In.
`*.native.jsx`	Platform-specific views (map, quiz) resolved automatically by the bundler for Android/iOS.

5.1 Game rules in code

Each continent has nine questions, numbered by `gameQuestion` along a geographic route. Only question 1 is unlocked at first. Answering the current question moves `unlockedQuestion` forward by one. Choices are the correct name plus three distractors from the same continent, shuffled with Fisher–Yates. Rewards are calculated once per species:

```
this.collection.feathers += 1; // one feather per new species
const discoveredCount = Object.keys(this.collection.discoveredBirds).length;
const eggsEarned = discoveredCount % 3 === 0 ? 1 : 0; // an egg every third discovery
```

Listing 1. Reward rule in `registerBirdDiscovery` (`gameModel.js`).

6 APIs and Services

Table 5. External APIs and services used by **Birdify**.

Service	Endpoint / SDK	Used for
Wikipedia REST API	`GET /api/rest_v1/page/summary/{title}`	Bird photograph (thumbnail) and a short description shown in "More about this bird".
xeno-canto API v3	`GET /api/3/recordings?query=sp:"{scientific name}"`	Real bird-song recordings for the audio clue, with recordist credit.
Firebase Authentication	`firebase/auth`	Email/password accounts with email verification; Google Sign-In credential exchange.
Cloud Firestore	`firebase/firestore`	One document per user (`birdifyGame/{uid}`) holding progress and collection.
Google Maps JavaScript API	`@react-google-maps/api`	Interactive route map on the web.
Google Maps SDK	`react-native-maps`	Native map with markers and polyline on Android/iOS.
Google Sign-In	`@react-native-google-signin`	Native Google account picker and ID token.

6.1 Fetching the clues

Both content APIs are called when a question loads. Images are looked up by *common* name, since Wikipedia article titles match common names. Audio is looked up by *scientific* name, which is unambiguous in xeno-canto's species filter.

```
export async function fetchBirdImage(commonName) {
  const url = `https://en.wikipedia.org/api/rest_v1/page/summary/${encodeURIComponent(commonName)}`
      ;
  const response = await fetch(url, { headers: { 'Api-User-Agent': 'Birdify/1.0' } });
  if (!response.ok) throw new Error(`Wikipedia error ${response.status} for "${commonName}"`);
  const data = await response.json();
  return { imageUrl: data.thumbnail?.source ?? null, wikiExtract: data.extract ?? "" };
}

export async function fetchBirdAudio(scientificName) {
  const query = encodeURIComponent(`sp:"${scientificName}"`);
  const url = `https://xeno-canto.org/api/3/recordings?query=${query}&key=${XENO_CANTO_API_KEY}`;
  const data = await (await fetch(url)).json();
  const recording = data.recordings?.[0];
  if (!recording) throw new Error(`No audio found for "${scientificName}"`);
  return { audioUrl: recording.file, audioCredit: `Recorded by ${recording.rec} · XC${recording.id}
      ` };
}
```

Listing 2. Clue fetching in `src/birdSource.js` (abridged).

The promises are stored in the model through a small `resolvePromise` helper that records `data` or `error`. The quiz view can then show a loading state ("Loading audio…") and fall back gracefully if one clue fails, without blocking the other.

6.2 Keys and configuration

All keys (Firebase, xeno-canto, Google Maps) are read from environment variables (`EXPO_PUBLIC_*`) defined in a git-ignored `.env`. A `.env.example` template is committed. The Android `google-services.json` is also excluded from version control.

7 Testing

7.1 API feasibility testing

Before any game code was written, we built a throw-away test screen in `src/app/index.jsx` to check that the two content APIs could support the concept:

- Searching a bird by name returned a Wikipedia summary, thumbnail and extract.
- A xeno-canto v3 query returned recordings that played on-device through `expo-av`.
- A country query (`grp:birds cnt:sweden`) with random paging confirmed that location-based sampling was possible.

This spike showed that exact English-name matching in xeno-canto was brittle. As a result, the final `birdSource.js` queries by scientific name.

7.2 Cross-platform functional testing

Every feature was tested by hand on three targets: the iOS Simulator, the Android Emulator (API 35) and desktop browsers. The main defects found and fixed are listed in Table 6.

Table 6. Selected defects found during testing and how they were resolved.

Issue observed	Resolution
Images and audio did not appear in the first prototype.	Fixed the loading of both clues in the prototype build (25 March).
Answer options reshuffled on every re-render, so choices moved under the user's finger.	Choices are now built once per question and stored in the model (`currentChoices`).
Button and card styles broke on Android.	Adjusted `StyleSheet` values so buttons and cards render the same on Android and the web.
Headings overlapped the status bar and notch on phones.	Wrapped screens in `SafeAreaView`.
Returning players restarted a continent from the beginning.	Levels now resume from the first unanswered question.
Achievements grid did not scroll on small screens.	Rebuilt the grid with a `FlatList`.
The web map showed "For development purposes only" and a Google Maps API error (Figure 5c).	Caused by the Google Maps key configuration; the README now documents that a properly configured, restricted Maps key is required.
Accidental sign-outs.	Added a sign-out confirmation dialog.

7.3 State and persistence testing

Persistence was tested by completing stops, force-closing the app and signing in again on a different platform, which confirmed that progress and collections synced through Firestore. We also tested with older documents that lacked newer fields; `getUnlockedQuestion()` falls back to question 1 so these accounts keep working. MobX strict mode (`enforceActions: "always"`) was enabled to catch any state change made outside an action.

> **Limitation.** Jest (`jest-expo`) is configured, but the project has no automated test suite yet. The pure helpers in `gameUtilities.js` and the reward logic in `gameModel.js` are the natural first candidates for unit tests.

8 User Testing

8.1 Method

Participants used the app on their own device and then filled in a 12-item questionnaire (Google Forms). Items covered navigation, layout on the participant's device, how easy targets were to find, visual hier-

archy, readability, information load, clarity of the quiz, and an overall *intuitiveness* rating on a 1–5 scale, plus open questions on favourite parts and suggested changes. Participants are anonymised as P1–P4.

Table 7. Test rounds.

Round	Date	Build	Participants
1 — Initial	26 March 2026	Prototype 1.0 (Europe only)	P1
2 — Final	11 May 2026	All continents, mainly Android	P2, P3, P4

8.2 Quantitative results

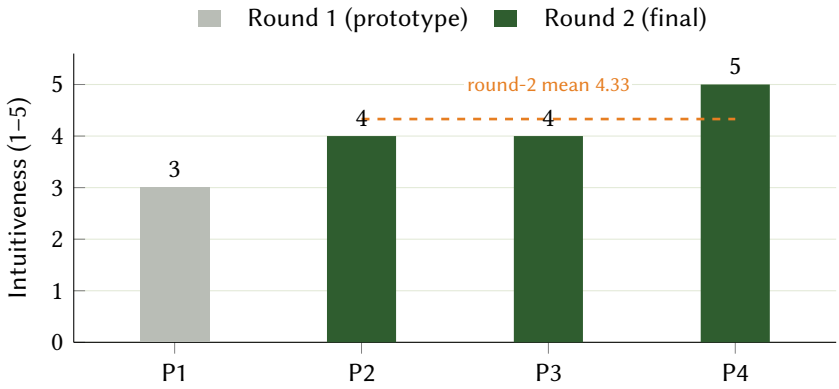


Figure 9. Answers to "Would you rate the layout as intuitive?". The mean rose from 3.0 (prototype) to 4.33 (final).

Table 8. Closed-question outcomes, coded from the free-text answers (✓ = positive, ~ = partial or with reservations, ✗ = negative).

	Round 1	Round 2		
Question	P1	P2	P3	P4
Navigation between screens is clear	✗	✓	✓	✓
Layout fits the device	✗	✓	✓	✓
Buttons easy to find and tap	~	✓	✓	✓
Sections easy to distinguish	~	✓	✓	~
Colours / hierarchy help focus	✓	✓	✓	✓
Text readable	✓	✓	✓	~
Not overwhelmed or confused	✓	✓	✓	✓
Clear what to do in the quiz	~	✓	✓	~

8.3 Qualitative findings

What participants liked. The bird-name quiz (P1), the continent layout (P2), and above all the **loading animation of the flying bird** and other animations (P2, P3, P4). The right/wrong colour feedback was singled out as clear (P2).

Round 1 — initial prototype (P1). Navigation was the main problem. The Home entry point was "very small" in a corner with no logo, and its purpose was unclear. The route lines on the map had

"no direction"; buttons gave no feedback on hover; the level button did not draw attention; a bird-name notice overlapped the back button; and the summary page showed the same picture twice.

Round 2 — final build (P2–P4). Feedback moved from "I can't find my way" to suggestions about the game itself:

> *"Probably a separate view for the collectables, that looked like badges. Enhance the game design to have challenges. A design to show that this is a tough level." — P4*

Other points: the collectible icons at the top were small and it was hard to tell which was which (P4); emoji were used for Home/Sign-out instead of icons (P4); the start control could be more prominent (P4); there should be one question per round instead of three at once (P2); more colour would help tell sections apart (P2); and one participant would rather navigate a world map than pick from a list (P3).

8.4 From feedback to features

Table 9 traces each piece of feedback to the change it led to.

Table 9. Feedback-driven changes.

Rd.	Feedback	Change implemented	Status
1	Direction of map route unclear	Dashed, styled route line; stops re-ordered to follow a geographic path	Done
1	No feedback on hover/press	Hover and press states and animations on all buttons; right/wrong Lottie animations	Done
1	Level button doesn't catch attention	Restyled, animated *Start challenge* button; orange "current" pin	Done
1	Home is small and its purpose unclear	Labelled Home button; onboarding panel "Here's how **Birdify** works" (Fig. 6)	Done
1	Duplicate image on summary; overlap with back button	Summary view reworked; `SafeAreaView` layout	Done
2	Separate collectibles view that looks like badges	**Achievements sheet with 13 badges** in three tracks, locked "???" states	Done
2	Enhance the game with challenges; show that a level is tough	**Gated route**: each stop is a challenge unlocked only by clearing the previous one, with padlocked markers and "Complete previous stops first"; trophy for a correct run	Done
2	One question per round, not three	Refactored to **one bird at a time**; progress shown as "Birds discovered *n*/9"	Done
2	Collectible icons small / unclear	Counter opens the Achievements sheet with labelled totals; in-app hints explain feathers and eggs	Done
2	More colour to separate sections	Continent cards fronted by bird photographs	Done
2	Emoji instead of icons for Home/Sign-out	—	Open
2	Navigate a world map instead of a list	—	Future

9 Conclusion and Future Work

Birdify went from an infographic and a sketch on black paper to a three-platform game with real photographs, real bird song, cloud-synced progress and an achievements system. The most valuable lesson came from user testing. The first round exposed basic navigation problems we had become blind to. Once those were fixed, the second round could focus on *game design*, and its suggestions (badges, gated challenges, one bird at a time) gave us the most engaging features of the final product.

Future work.

- A zoomable world map as the entry point, replacing the continent list (P3).
- A vector icon set in place of emoji for navigation controls (P4).
- Explicit difficulty tiers, e.g. audio-only questions or look-alike distractors for later stops.
- An automated unit-test suite for the model and helpers, and a larger, more diverse round of usability testing.